

Drone Control Simulation

Lirui Wang

Abstract—This is the writeup for EE 447 final project. The goal is to design a system that attempts to control the SE2 movement for a quadrotor drone. In this work, we deploy a simplified math model and use time-domain and frequency-domain techniques to build the controller.

I. INTRODUCTION

Quadrotor that is outfitted with camera has been played a critical role in entertainment, work and military usages. Since the quadrotors have rotational symmetry, a 2D model of a quadcopter moving in a plane suffices to cover most of the applications. Reference horizontal and vertical position are often provided by the consumers and the onboard depth camera will provide (nonlinear) measurements of the horizontal and vertical position and the rotation angles. The task of the controller is to compute the thrust command that will stabilize the system. We also provide a decent robust system that strives for improving upon single, double and triple flips in real world. To design and simulate the controller of the quadrotor, we present a controller with expected performance, sensitivity, and robustness properties and full simulation with reproducibility. The frequency PID controller has a stability margin above 1, gain margin above 2 and phase margin above 60 degree. By its convergence in simulation, we expect users to control the quadrotor with stability and robustness.

II. MATHEMATICAL MODEL

We introduce the state space model of the quadrotor in this section. The 6-dimensional system state consists of the quadrotor's rotation q_θ and (horizontal and vertical) position (q_H, q_V) in this plane, as well as the time derivatives $(\dot{q}_\theta, \dot{q}_H, \dot{q}_V)$. In addition, we have the nonlinear relationships given by

$$I\ddot{q}_\theta = \tau_R - \tau_L$$

$$m\ddot{q}_H = (\tau_R + \tau_L) \sin q_\theta - \beta \dot{q}_H$$

$$m\ddot{q}_V = -mg + (\tau_R + \tau_L) \cos q_\theta - \beta \dot{q}_V$$

So, the state space form of the mathematical model

representation is $x = \begin{bmatrix} q_\theta \\ \dot{q}_\theta \\ q_H \\ \dot{q}_H \\ q_V \\ \dot{q}_V \end{bmatrix}$. The controller thrust commands

for the left- and right-side motors is $u = \begin{bmatrix} \tau_L \\ \tau_R \end{bmatrix}$ and

measurements of the horizontal and vertical position from camera $r = \begin{bmatrix} r_H \\ r_V \end{bmatrix}$ $y = \begin{bmatrix} q_H \\ q_V \\ q_\theta \end{bmatrix}$ and nonlinear dynamic system (denoted as NL)

$$f(x) = \dot{x} = \begin{bmatrix} \dot{q}_\theta \\ (\tau_R - \tau_L)/I \\ \dot{q}_H \\ ((\tau_R + \tau_L) \sin(q_\theta) - \beta \dot{q}_H)/m \\ \dot{q}_V \\ (-m \cdot g + (\tau_R + \tau_L) \cos(q_\theta) - \beta \dot{q}_V)/m \end{bmatrix}$$

The expected output in terms of the reference point should be

$$y = \begin{bmatrix} r_H \\ r_V \\ 0 \end{bmatrix} \text{ with no rotation bias. Specifically, in the simplified model, the constants used are } m = 1\text{kg}, I = 0.01\text{kg/m}^2, g = 9.81\text{m/sec}^{-2}, \beta = .1\text{kg/sec}^{-1}.$$

A. Interpretation of the Models

As shown in the schematic model, assume the quadrotor is at some position (q_V, q_H) of the plane with rotation q_θ . We are given a reference position that consumer wants to reach, and our onboard depth camera is able to provide the displacement and rotations relative to the origin as output. The controller will decide the control input such that the input would help the quadrotor converge to the reference by feedback. We assume that we are controlling the quadrotor on a plane, so we have two dimension rotation and one dimension rotation. For rotation, $I\ddot{q}_\theta = \tau_R - \tau_L$ come from the relationship between torque and angular acceleration. The quadrotor would turn rotation based on the difference between the thrust from left and right actuator. For horizontal dynamics, $m\ddot{q}_H = (\tau_R + \tau_L) \sin q_\theta - \beta \dot{q}_H$ comes from the Newton's force conservation, that the horizontal acceleration can be calculated by subtracting the air frictional force from the projected horizontal force from thrust. Similarly, the vertical acceleration can be calculated by subtracting the sum of air frictional force and gravitational force from the projected vertical force from thrust $m\ddot{q}_V = -mg + (\tau_R + \tau_L) \cos q_\theta - \beta \dot{q}_V$. Furthermore, the consumer's requirement is simplified to be only the destination of the reference positions. However, it's assumed that we want the quadrotor to stay upright at the reference position. Also, our simplified first principled model only approximates the complexity of the dynamics with nonlinearity such as $\cos$ and $\sin$. Moreover, the specific statistics used in the model is simplified and probably doesn't have much relation to physical world. All the environmental signals and variables are ignored such as air, battery and hardware. We As a mathematical model with simulation, if $\lim_{t \rightarrow \infty} (q_V, q_H, q_\theta) \rightarrow (r_V, r_H, r_\theta)$, then our quadrotor is guaranteed to stabilize on the plane.

B. Operating Point and Stability Analysis

The operating point is $(r_V, r_H) = (4, 4)$, which is the same as the set reference position. The reference height is set to

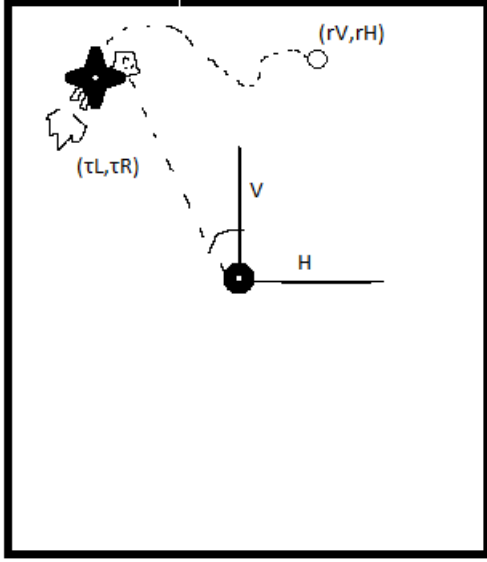


Fig. 1: Schematics for the Physical Model

be 4(meter) for which we expect that consumers would like to view the quadrotor while leaving it stay at a safe position from the ground. The rotation q_θ is fixed to be 0 because we don't want the final position of quadrotor is biased to any direction. Additionally, all the 1st and 2nd derivative (velocity and acceleration) should be 0 at this operating point such that quadrotor can hover on that position without unstable adjustment. In total, our numerical method calculation

$$\text{calculates the equilibrium state to be } x_e = \begin{bmatrix} 0 \\ 0 \\ 4 \\ 0 \\ 4 \\ 0 \end{bmatrix} \quad u_e = \begin{bmatrix} g/2 \\ g/2 \end{bmatrix}.$$

This would match with the math calculation: to have

$$\ddot{q}_\theta = 0 \rightarrow I\ddot{q}_\theta = \tau_R - \tau_L = 0 \rightarrow \tau_L = \tau_R$$

$$\ddot{q}_H = 0, \dot{q}_H = 0 \rightarrow m\ddot{q}_H = (\tau_R + \tau_L) \sin q_\theta - \beta\dot{q}_H = 0$$

$$\rightarrow \tau_R = \tau_L \text{ or } q_\theta = 2k\pi$$

$$m\ddot{q}_V = -mg + (\tau_R + \tau_L) \cos q_\theta - \beta\dot{q}_V = 0$$

$$\rightarrow \tau_R = \tau_L \text{ and } \tau_L + \tau_R = mg$$

The equilibrium input from the controller should require both motor to give equal thrust force which sums up to be the gravity force, which exactly interprets the case where the quadrotor is able to hover upright on the position.

1) *Linearization*: We linearize the original nonlinear system on the operating point (x_e, u_e) using numerical methods. The Jacobian matrix for the state spaces representation can be

calculated as follows: (Take matrix A for instance)

$$A = \begin{bmatrix} A_{11} & \dots & \dots & A_{16} \\ A_{21} & \dots & \dots & A_{26} \\ \dots & \dots & \dots & \dots \\ A_{61} & \dots & \dots & A_{66} \end{bmatrix} \quad \text{where}$$

$$A_{jk} = \left. \frac{\partial f_j}{\partial x_k} \right|_{(x_e, u_e)} = \lim_{\Delta \rightarrow 0} \frac{1}{\Delta} (f_j(x_e + \Delta e_k, u_e) - f_j(x_e, u_e))$$

$$\approx \frac{1}{\Delta} (f_j(x_e + \Delta e_k, u_e) - f_j(x_e, u_e))$$

So, A is a 6x6 matrix, B is a 6x2 matrix, C is a 2x6 matrix and D is a 2x2 matrix. We denote the state, input, and output of (LTI) system using ξ, μ, η , relative to the equilibrium point. Then, the full state space representation of our model is

$$\dot{\xi} = A\xi + B\mu$$

$$\eta = C\xi + D\mu$$

. Moreover, we obtain the relationship between (LTI) and (NL) systems $x \approx x_e + \xi$ and $y \approx y_e + \eta$ when $u = u_e + \mu$.

2) *Stability*: We assess the stability of this model in state space. For simplicity, consider $\dot{x} = Ax$, and we can determine the stability of the origin by the set of eigenvalues of A. In other words, the characteristic polynomial $\det(SI - A) = 0$ is the equation that we are interested in. This equation solely determines if the system would be stable around the origin. From the computation, it turns out that there are only two distinct roots $s_1 = -0.1, s_2 = 0$, so we know that the system is neutrally stable. However, we are expecting asymptotically stable system such that the quadrotor would be able to converge from some perturbed initial point to the equilibrium. Figure 2 shows the initial divergence situation of the vertical and horizontal positions of the quadrotor before any control input is applied.

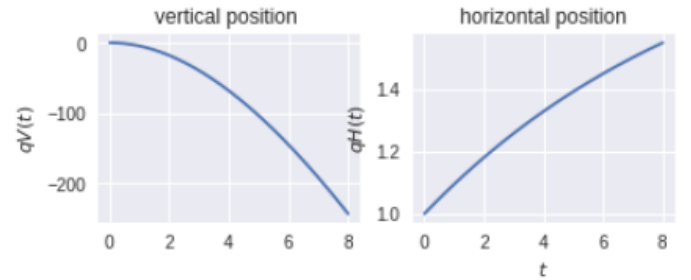


Fig. 2: Initial Divergence (NL)

C. State Space Controller Design

In this section, we discuss our methods of controlling the nonlinear model assuming full state observability. We synthesize a closed-loop state feedback law that asymptotically stabilizes (LTI) using eigenvalue assignment. Then, we synthesize a state estimator (i.e. an observer) that asymptotically estimates the state of (LTI) using eigenvalue assignment. We also implement the state feedback law to control LTI

using observer's state. Lastly, we apply the closed-loop estimator/controller to the nonlinear model and observe that it also successfully stabilizes nonlinear model well given close enough initial point.

1) *Close-loop State Feedback Controller*: Controlling a LTI system on state space involves gain matrix K , such that the controller input to the system is $\mu = -K\xi$ as a negative feedback to encourage convergence to the origin. So, our new dynamics becomes $\dot{\xi} = (A - BK)\xi$. Now, similar to the discussion above, we are interested in the poles of the characteristic equation, which corresponds to the eigenvalues of matrix $(A - BK) : \det(SI - (A - BK))$. Since the poles should be placed at the left side of the complex plane, we manually choose the poles and obtain the gain matrix K by pole placements of the control toolbox. As shown in figure 3, we successfully control the LTI system to asymptotically converge to the origin. This result gives a more realistic and reliable quadrotor performance. Imagine in the real world context, given a command to return to the ground position $O : (0, 0)$ and some initial perturbation, our linearized system would not only keep close to the ground, but it also lands on the points it takes off. For quadrotor, a successful landing signal sent from the onboard camera would provide useful information to control its propellers and battery (a full stop position) and prevents consumer from damaging the quadrotor or endangering themselves.

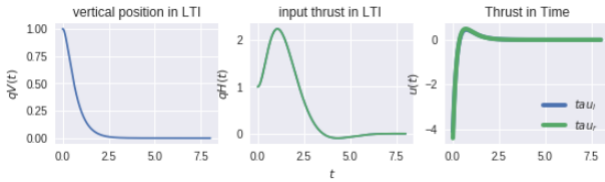


Fig. 3: LTI system convergence about the origin

2) *State Estimator*: Now assume we don't have full state information, only the outputs are observable. We attempt to construct another LTI system that would help us estimate the states of the real system. This is a realistic problem as the onboard camera only provides us the angle of the quadrotor position and its actual position, rotational and translational velocity are unknown. However, given the observed angle in the observer system and the angle in the real system, we can simulate our observer system to converge to the real system by pole placements techniques. Formally, we define observer state and output

$$\dot{\hat{\xi}} = A\hat{\xi} + Bu + L(\eta - \hat{\eta})$$

$$\hat{\eta} = C\hat{\xi} + Du$$

. In addition, we define $e = \xi - \hat{\xi}$ as the error between the state estimator and the real system. This yields the error equation $\dot{e} = (A - LC)e$. So, we can apply pole placement for its characteristic equation to stabilize the error. As the eigenvalues of $A - LC$ all stay in the left plane, the error states converge to 0 and the estimator asymptotically estimate the state of

(LTI). Closing the loop, we have the augmented state space representation

$$\frac{d}{dt} \begin{bmatrix} x \\ e \end{bmatrix} = \begin{bmatrix} A - BK & BK \\ 0 & A - LC \end{bmatrix} \begin{bmatrix} x \\ e \end{bmatrix}$$

. Remarkably the system is block diagonal, so its characteristic polynomial factors into the product $a(s) = \det(SI - (A - BK)) * \det((SI - (A - LC)))$, which provides the basis for us to design the full state feedback controller and estimator separately. The graphs for the error convergence in both time domain and state space are shown in Fig 4 and the plots for state estimator for horizontal and vertical positions are shown in Fig 5. This is different than the LTI system convergence because we are not using the real full state information to do the control feedback. Instead, we use the observer state as an estimation, which would eventually converge to the real state by pole placement. After that we use estimated state as information for negative feedback to control the real system. So, it's expected that the convergence speed would slow down by a ignorable amount.

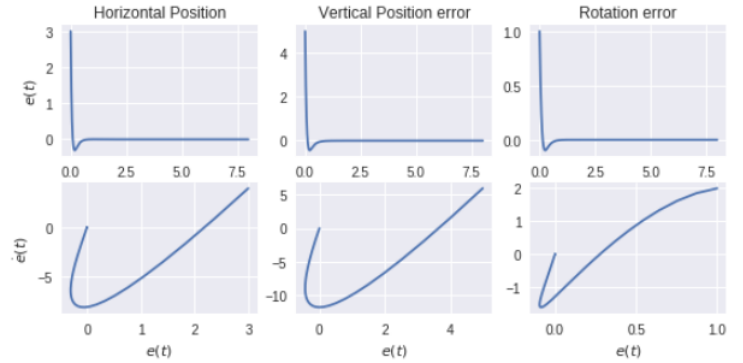


Fig. 4: Error Convergence in Time and State Space

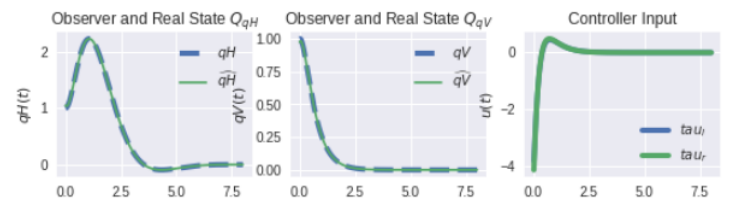


Fig. 5: Observer Convergence about the Origin

3) *Closed loop estimator/controller on NL model*: Before applying the estimator and feedback controller in nonlinear model, we need to point out that these techniques using Jacobian are first order methods, so they only work well if the initial state is close enough to the equilibrium. In other words, we are approximating the nonlinear function using a linear controller at the equilibrium, so the controller performance is limited by the approximation. Recall that $x \approx x_e + \xi$ and $y \approx y_e + \eta$ when $u = u_e + \mu$, so our new equations need to take

this offset into account. The synthesized observer/controller should have the expression:

$$\begin{aligned}\mu &= -K(\hat{\xi} - x_e) + u_e \\ \eta &= C(\hat{\xi} - x_e) + D(\mu - u_e) \\ \dot{\xi} &= A(\hat{\xi} - x_e) + B(\mu - u_e) + L(\eta - \hat{\eta})\end{aligned}$$

Note that the resulting trajectory successfully controls the nonlinear model to converge to the reference position (Fig 6). The initial perturbation is chosen to be $0.05 * \mathcal{N}(1, 1)$. Note that compared with the observer controller about the origin, it now converges to the reference point. The input also converges to half of the gravity force. While it still assumes no full state information, it provides control to the nonlinear model, which is closer to the real-world situations. Based on the plot, its convergence seems to be slower since the NL model is more complex.

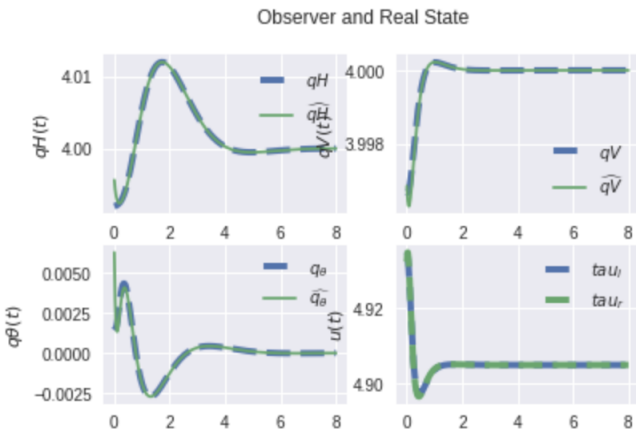


Fig. 6: Observer Convergence about the Equilibria for NL

D. Frequency Analysis

In this section, we discuss a different model and use frequency domain techniques to analyze and design a controller. Besides position requirement, we are interested in the rotation dynamics of the quadrotor, so we have the following simplified control task: $I\ddot{q}_\theta = \tau$ where $u = \tau$ is input and $y = q_\theta$ is output. Treating transfer function as a black box, we take the input $u = e^{st}$ with frequency s , and its output has the form $y = G(s)e^{st}$. So, we analyze the transfer function of this model in frequency domain to capture the steady state performance. For this LTI system, we have two ways to calculate the transfer function. Like above, we can calculate the state space form A, B, C, D and compute transfer function using $\det(C(SI - A)^{-1} * B + D)$ or we can integrate $I\ddot{q}_\theta = \tau$ twice, and get the expression for $y(t) = 1/(Is^2)u(t)$. So, we have the processor model transfer function $P(s) = 1/(Is^2)$. For controller comparisons, we provide the root locus of using proportional controller, PI controller and PID controller. After some hand tuning and heuristics, the ranges of proportional, integral and derivative are fixed to be $[0, 0.6], [0, 0.3], [0, 0.1]$. From the graph, PID

controller brings all the poles to the left plane so we use it in the experiment: $k_p = 0.5, k_i = 0.2, k_d = 0.1$. The transfer function for controller is $C = 0.5 + \frac{0.2}{s} + 0.1s$. We decided to deploy PID controller to maximize the stability of the model. A root locus is shown in Fig 7. To analyze using Nyquist

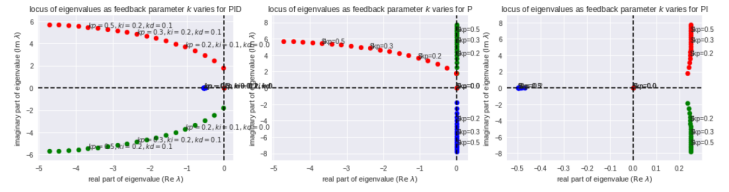


Fig. 7: Root Locus of Transfer Function (PID)

criterion, we calculate the open loop transfer function

$$L = P * C = \frac{100s(0.1s + 0.5) + 20.0}{s^3}$$

The plotted Nyquist plot of our transfer function turns out to be far away from the critical point -1 . By Nyquist criterion, the closed loop system is stable if and only if the closed loop contour $\Omega = \{L(i\omega) : -\infty < \omega < \infty\}$ given by has no net encirclements of the critical point $s = -1$ [1], which is clearly the case. Similar to analyze in cartesian coordinates, we can use polar coordinates to analyze the Bode plot for more details about the phase and amplitude of the transfer function in frequency domain. From the bode plot, we know the PID controller is very robust to noise, the amplitude stays far away from the 1 and the only slightly reacts to low frequency noises. Although its phase crosses 180 degree, it keeps increasing and stays stable at around 270 degrees. We introduce the sensitivity

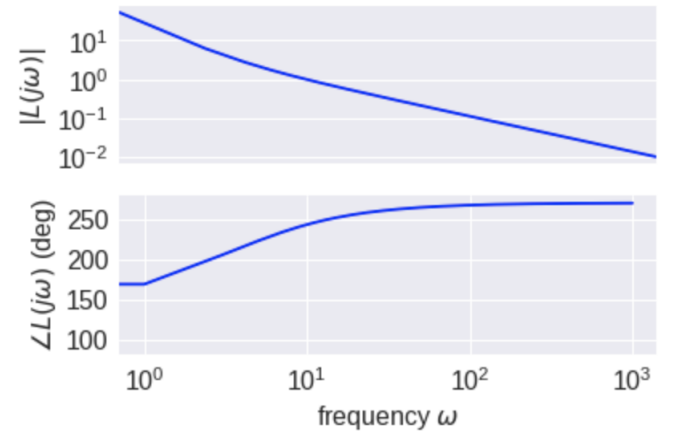


Fig. 8: Bode Plot of Transfer Function

function as $S(j\omega) = \frac{1}{1+L}$, which would be the basis of our robustness to the output disturbance to the output (Fig 9). We define stability margin s_m as the shortest distance from $S(j\omega)$ to critical point, gain margin g_m as the smallest increase in gain that creates an encirclement, and the phase margin ϕ_m as the smallest increase in phase that creates an encirclement [1]. The values are determined to be $s_m = 1, g_m = 2, \phi_m = 63$ (Fig 10), which implies that our PID controller is very robust and insensitive to disturbance. For valid ranges of frequency,

the phase angle starts above 180 degrees and increase, the amplitude drops from 1 in the bode plot. Note that the transfer function has more poles than zeros, so its value converge to zero as s increases. Based on the Nyquist plot of the loop transfer function From the annotated stability margin, we can also conclude that the open loop transfer function is distant from instability $(-180, 1)$.

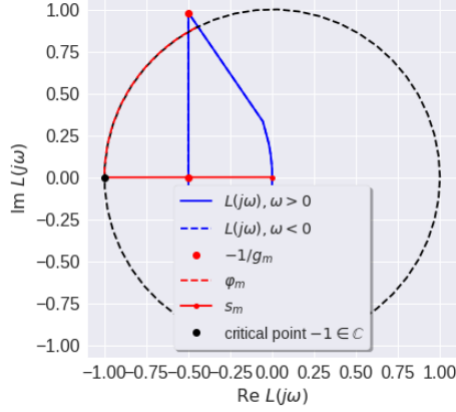


Fig. 9: Nyquist Plot with Stability Margins

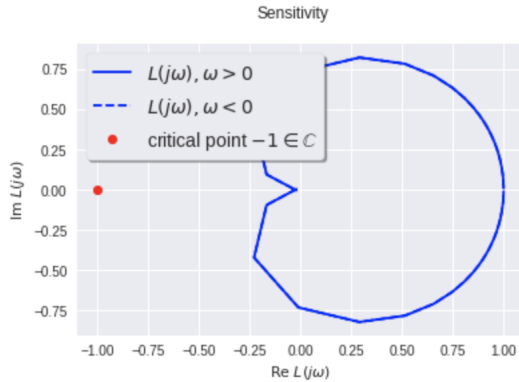


Fig. 10: Sensitivity Function Plot

III. DISCUSSION

A. Summary

In this work, we mainly designs a controller from time and frequency domain to stabilize a quadrotor given a reference position. We discuss a simplified (NL) model and linearize it around the origin. We use pole placements to design feedback controller to stabilize the system. After that we assume state is unobservable, and design a state estimator with controller and successfully control the NL system using the LTI observer. Moreover, we work on rotational dynamics individually using frequency domain techniques. We design a PID controller using root locus method, analyze the nyquist, bode and sensitivity plot of the transfer function. F In conclusion, our model is supposed to control both the rotation and translation (q_V, q_H, q_θ) well for this specific simplified model in LTI

domain. For NL, we require the initial state to be close to the equilibria such that our first order method can apply. For frequency method, our stability margins and wide frequency range analysis offers strong foundations for the robustness. For all these experiments, we run simulation on a valid, wide range of possible states and explain the performance in the context above.

B. Extension

Admittedly, there are various factors that are not discussed or considered in our model. In terms of the model, in real world, we have more complex models, more environment and control requirements involved, which might augment the state space. The environment issues might introduce statistical noises to our model, and we need to take those into account by methods such as Monte Carlo methods. For a Hidden Markov Model, we can no longer guarantee that the quadrotor would move as our input says. Similarly we did not cover the filtering of the sensor information and assume the accuracy. Moreover, we might have a optimal control problem where we want to minimize the costs, for instance battery, such that the quadrotor can stay longer in the air. Alternatively, we instead work on a search problem in the state space and action space where we want to find the shortest path to the reference position. Then more regressive controller such as LQR and estimator such as Kalman Filter would be applied to achieve the goal. When these requirements and ideas augment our state representation, our control system would have to be redesigned. Lastly, before touching the actual physical system and implement the controller physically on the embedded systems, numerous experiments on the simulations should be conducted.

REFERENCES

- [1] K.Astrom and R. Murray. Feedback Systems: An Introduction for Scientists and Engineers 2008, Princeton University Press, Princeton, NJ, USA.